



Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias Físico Matemáticas

Administración Integral de Riesgo

**Fundamentos de modelos de
Machine Learning**

Gustavo Rojas López
Milton Escobar Trujillo
Alexis Gimar Isidro Meza
Ulises Galicia Flores

Índice

Índice	1	10.1. Función de Pérdida	10
		10.2. Derivación Completa de Backpropagation	10
		10.3. Gradiente Evanesciente y Explosivo	10
Random Forest	3	11. Optimización	11
1. Introducción	3	11.1. Descenso por Gradiente y Variantes	11
2. Fundamentos Teóricos	3	11.2. Momentum y Adam	11
2.1. Definición Formal del Árbol de Decisión	3	12. Supuestos y Propiedades	11
2.2. Medidas de Impureza	3	12.1. Capacidad y Descomposición Sesgo-Varianza	11
3. Construcción Matemática	4	12.2. Regularización	11
3.1. Bagging	4	13. Aspectos Computacionales	12
3.2. Selección Aleatoria de Variables	4	13.1. Complejidad y Paralelismo en GPU	12
3.3. Estimador Agregado	4	13.2. Inicialización de Pesos	12
3.4. Reducción de Varianza	4	14. Implementación en Python	12
4. Supuestos y Propiedades	4	14.1. Uso en PyTorch	12
4.1. Descomposición Sesgo-Varianza	5	14.2. Ejemplo	12
4.2. Consistencia del Estimador	5	14.3. Ajuste de Hiperparámetros	14
4.3. Robustez frente a Sobreajuste	5	15. Arquitecturas Canónicas	14
5. Aspectos Computacionales	5	15.1. Redes Convolucionales	14
5.1. Complejidad del Algoritmo	5	15.2. Redes Recurrentes y LSTM	14
5.2. Estimación OOB y Grandes Datasets	5	eXtreme Gradient Boosting	15
6. Implementación en Python	5	16. Introducción	15
6.1. Uso en scikit-learn	5	17. Fundamentos Teóricos	15
6.2. Ejemplo	5	17.1. Aprendizaje Estadístico Aditivo	15
6.3. Ajuste de Hiperparámetros	6	17.2. Optimización de la Función Objetivo	15
7. Importancia de Variables y Extensiones	6	18. Construcción matemática	15
7.1. Importancia por Disminución de Impureza (MDI)	7	18.1. Expansión de Taylor de la Función de Pérdida	15
7.2. Variantes y Extensiones	7	18.2. Determinación de Pesos Óptimos	15
Redes Neuronales	8	18.3. Algoritmo de División (Gain)	16
8. Introducción	8	19. Supuestos y Propiedades	16
9. Fundamentos Teóricos	8	19.1. Diferenciabilidad de la Función de Pérdida	16
9.1. La Neurona Artificial	8	19.2. Tratamiento de Valores Faltantes (Sparsity Awareness)	16
9.2. Funciones de Activación	8	19.3. Control de Sobreajuste mediante Regularización	16
9.3. Red Feedforward como Composición de Funciones	9	19.4. Invarianza a la Escala	16
9.4. Teorema de Aproximación Universal	9	20. Aspectos computacionales	16
10. Construcción Matemática	10		

20.1. Aprendizaje en Paralelo y Estructura de Bloques	16
20.2. Algoritmo de Cuantiles Aproximados	16
20.3. Optimización de Caché y Computación Out-of-Core	17
21. Interpretación y Atribución de Variables	17
21.1. Puntuación de Importancia (F-score)	17
21.2. Valores SHAP y Explicabilidad Local	17
22. Control de Sobreajuste mediante Parada Temprana	17
23. Implementación en Python	17
23.1. Configuración de hiperparámetros .	18
Análisis Comparativo de Modelos	19
24. Síntesis Metodológica	19
25. Referencias	21

Random Forest

1. Introducción

Un árbol de decisión es un modelo de aprendizaje automático más que consta formalmente de dividir recursivamente el espacio de características en regiones rectangulares y asigna una predicción constante a cada región.

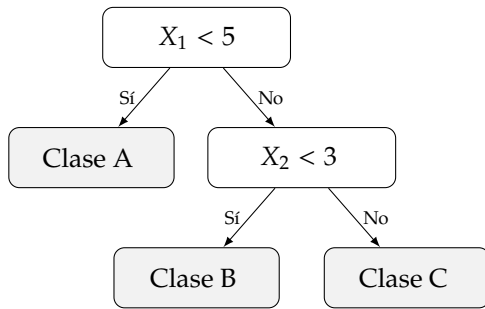


Figura 1: Ejemplo ilustrativo de un árbol de decisión binario.

Sin embargo, los árboles profundos muestran una varianza extremadamente alta, es decir pequeñas perturbaciones en los datos de entrenamiento producen estructuras completamente diferentes, lo que conduce a un sobreajuste severo y a una mala generalización.

Random Forest resuelve este problema combinando dos ideas, primeramente el bagging (*bootstrap aggregation*), cuya función es promediar varios modelos entrenados en subconjuntos de datos generados por remuestreo, y la selección aleatoria de variables, que decorrelaciona los árboles individuales para maximizar la reducción de varianza del conjunto. El resultado es un estimador con varianza sustancialmente menor que la de cualquier árbol individual, sin incrementar significativamente el sesgo, lo que lo convierte en uno de los algoritmos más competitivos y robustos en la práctica.

Random Forest es especialmente adecuado para datos tabulares con variables numéricas y categóricas, donde existen relaciones no lineales y posibles interacciones entre características.

2. Fundamentos Teóricos

2.1. Definición Formal del Árbol de Decisión

Sea $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ con $\mathbf{x}_i \in \mathbb{R}^p$ e $y_i \in \mathbb{R}$ (regresión) o $y_i \in \{1, \dots, K\}$ (clasificación). Un árbol de decisión T define una partición $\{R_1, \dots, R_J\}$ del

espacio $\mathbb{R}^p$ tal que

$$\bigcup_{j=1}^J R_j = \mathbb{R}^p,$$

y asigna a cada región R_j un valor constante c_j . El predictor resultante es:

$$f_T(\mathbf{x}) = \sum_{j=1}^J c_j \mathbf{1}[\mathbf{x} \in R_j].$$

Las regiones se construyen mediante divisiones binarias de la forma $\{x_k \leq s\}$ vs. $\{x_k > s\}$ para la variable k y umbral s . El valor óptimo c_j en cada región minimiza la pérdida cuadrática, que resulta en $c_j = \bar{y}_{R_j}$ (media de las respuestas en R_j) para regresión, y $c_j = \arg \max_k |\{i : \mathbf{x}_i \in R_j, y_i = k\}|$ (clase mayoritaria) para clasificación.

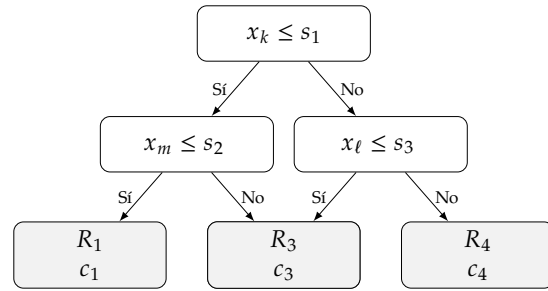


Figura 2: Árbol de decisión que induce una partición jerárquica del espacio $\mathbb{R}^p$.

2.2. Medidas de Impureza

La calidad de una división se mide mediante una función de impureza $Q(R)$ que cuantifica la heterogeneidad de las etiquetas en la región R . Las dos más empleadas son:

Impureza de Gini. Para una región con proporciones de clase $\hat{p}_k = |\{i \in R : y_i = k\}|/|R|$:

$$G(R) = \sum_{k=1}^K \hat{p}_k (1 - \hat{p}_k) = 1 - \sum_{k=1}^K \hat{p}_k^2.$$

$G(R) = 0$ cuando la región es pura (una sola clase), su máximo $1 - 1/K$ corresponde a distribución uniforme entre clases.

Entropía de Shannon.

$$H(R) = - \sum_{k=1}^K \hat{p}_k \log_2(\hat{p}_k),$$

con la convención $0 \log 0 = 0$. $H(R) = 0$ para nodos puros y $H(R) = \log_2 K$ para distribución uniforme.

La **ganancia de información** de dividir la región R en R_L y R_R es:

$$\Delta Q = Q(R) - \frac{|R_L|}{|R|} Q(R_L) - \frac{|R_R|}{|R|} Q(R_R),$$

y el algoritmo selecciona la división (k^*, s^*) que maximiza ΔQ . Para regresión, $Q(R) = \frac{1}{|R|} \sum_{i \in R} (y_i - \bar{y}_R)^2$ (varianza empírica), y la reducción de varianza reemplaza a la ganancia de información.

3. Construcción Matemática

3.1. Bagging

El método de *bootstrap aggregation* parte de la idea de que si dispusiéramos de B muestras i.i.d. de tamaño n provenientes de la distribución P , podríamos construir B modelos independientes $f_1, \dots, f_B$ y agregar sus predicciones. El estimador promediado tendría varianza B veces menor que la de un solo modelo (bajo independencia). Como no se dispone de varias muestras, se aproxima este esquema mediante remuestreo bootstrap.

Propiedad 1 (Muestreo Bootstrap). *Dada la muestra $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, una muestra bootstrap $\mathcal{D}^{*b}$ se obtiene seleccionando n observaciones con reemplazo de $\mathcal{D}$. La probabilidad de que la observación i sea incluida al menos una vez es:*

$$P(\text{incluida}) = 1 - \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} 1 - e^{-1} \approx 0.632.$$

En consecuencia, el $\approx 36.8\%$ de las observaciones originales quedan fuera de cada muestra bootstrap (denominadas out-of-bag, OOB).

Las observaciones OOB constituyen un conjunto de validación natural para cada árbol, permitiendo estimar el error de generalización sin necesidad de validación cruzada explícita.

3.2. Selección Aleatoria de Variables

La innovación central de Random Forest respecto al bagging puro consiste en que, en cada división de cada árbol, solo se considera un subconjunto aleatorio de $m \leq p$ variables candidatas. Formalmente, en el nodo t del árbol b , se muestrea sin reemplazo un índice aleatorio $\mathcal{M}_t^b \subset \{1, \dots, p\}$ con $|\mathcal{M}_t^b| = m$, y la división óptima se busca solo sobre:

$$(k^*, s^*) = \arg \max_{k \in \mathcal{M}_t^b, s \in \mathbb{R}} \Delta Q(R_t; k, s).$$

Los valores habituales son $m = \lfloor \sqrt{p} \rfloor$ para clasificación y $m = \lfloor p/3 \rfloor$ para regresión. Este mecanismo decorrelaciona los árboles, incluso si existe una variable dominante, no aparecerá en todas las divisiones de todos los árboles, forzando al bosque a explotar información distribuida entre múltiples variables.

3.3. Estimador Agregado

Sea $f_b(\mathbf{x})$ la predicción del b -ésimo árbol entrenado sobre $\mathcal{D}^{*b}$ con selección aleatoria de m variables. El estimador de Random Forest es:

Regresión:

$$\hat{f}_{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B f_b(\mathbf{x}).$$

Clasificación (votación mayoritaria):

$$\hat{y}_{RF}(\mathbf{x}) = \arg \max_{k \in \{1, \dots, K\}} \sum_{b=1}^B \mathbf{1}[f_b(\mathbf{x}) = k].$$

Equivalentemente, las probabilidades de clase pueden estimarse como $\hat{p}_k(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \mathbf{1}[f_b(\mathbf{x}) = k]$, y la predicción es $\hat{y}_{RF} = \arg \max_k \hat{p}_k(\mathbf{x})$.

3.4. Reducción de Varianza

La reducción de varianza es el fenómeno teórico central de Random Forest. Consideremos el modelo de regresión y denotemos por ρ la correlación entre dos árboles cualesquiera, σ^2 la varianza de un árbol individual y $\mu = \mathbb{E}[f_b(\mathbf{x})]$ su esperanza. La varianza del estimador promediado satisface:

$$\begin{aligned} \text{Var}[\hat{f}_{RF}(\mathbf{x})] &= \text{Var}\left[\frac{1}{B} \sum_{b=1}^B f_b(\mathbf{x})\right] \\ &= \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2. \end{aligned} \quad (3.1)$$

La descomposición (3.1) revela dos términos:

- $\rho \sigma^2$: *varianza irreducible*, proporcional a la correlación promedio entre árboles. No disminuye al aumentar B .
- $(1-\rho)\sigma^2/B$: *varianza reducible*, que tiende a cero conforme $B \rightarrow \infty$.

Intuición 1. *Para maximizar la reducción de varianza se desea ρ pequeño y B grande. El bagging reduce ρ respecto a los árboles deterministas, y la selección aleatoria de variables lo reduce aún más, pues cada árbol explota un subespacio diferente de variables. Sin embargo, reducir m incrementa el sesgo de cada árbol individual, por lo que existe un equilibrio óptimo en m .*

En el límite $B \rightarrow \infty$, la varianza converge a $\rho \sigma^2$, que es la varianza de la distribución del proceso aleatorio que genera los árboles. Por tanto, agregar más árboles siempre reduce el error (o lo mantiene), nunca lo incrementa, lo que hace al método monótonamente beneficioso respecto a B .

4. Supuestos y Propiedades

4.1. Descomposición Sesgo-Varianza

El error cuadrático medio de $\hat{f}_{RF}(\mathbf{x})$ en un punto $\mathbf{x}$ admite la descomposición clásica:

$$\mathbb{E}[(y - \hat{f}_{RF})^2] = \text{Bias}^2[\hat{f}_{RF}] + \text{Var}[\hat{f}_{RF}] + \sigma_\varepsilon^2,$$

donde $\sigma_\varepsilon^2 = \text{Var}[\varepsilon]$ es el error irreducible del proceso generador de datos. El sesgo de $\hat{f}_{RF}$ es aproximadamente igual al sesgo de un árbol individual, puesto que el promedio de estimadores insesgados es insesgado: $\mathbb{E}[\hat{f}_{RF}] = \mathbb{E}[f_b]$. Los árboles sin restricción de profundidad tienen sesgo prácticamente nulo pero varianza alta; el bosque preserva el bajo sesgo mientras reduce drásticamente la varianza mediante la fórmula (3.1).

4.2. Consistencia del Estimador

Bajo condiciones generales de regularidad (función objetivo acotada, crecimiento del número de árboles y profundidad máxima con n), el estimador de Random Forest es consistente: para toda $\varepsilon > 0$,

$$P(|\hat{f}_{RF,n}(\mathbf{x}) - f^*(\mathbf{x})| > \varepsilon) \xrightarrow{n \rightarrow \infty} 0,$$

donde f^* es la función de regresión óptima (o el clasificador de Bayes en clasificación).

4.3. Robustez frente a Sobreajuste

Una propiedad notable de Random Forest es su resistencia empírica al sobreajuste al incrementar B . La cota de generalización de Breiman establece que el error de generalización del bosque está acotado superiormente por:

$$\text{Error}_{gen} \leq \frac{\bar{\rho}(1 - s^2)}{s^2},$$

donde $\bar{\rho}$ es la correlación promedio entre árboles y $s = \mathbb{E}[\text{margen}]$ es la “solidez” del margen de clasificación. Esta cota muestra que el error de generalización *no crece* con B , es decir agregar árboles no daña la generalización, a diferencia de lo que ocurre con un solo árbol sobreajustado.

5. Aspectos Computacionales

5.1. Complejidad del Algoritmo

La construcción de un árbol de profundidad d sobre n observaciones con p variables tiene costo $O(np \log n)$ en el modo exacto (ordenar cada variable y evaluar todos los umbrales). Con la selección de m variables por nodo, el costo se reduce a $O(nm \log n)$ por árbol. Para el bosque completo de B árboles:

$$\text{Complejidad total} = O(B \cdot n \cdot m \cdot \log n).$$

La predicción de una observación nueva recorre B árboles, cada uno en $O(d)$, con costo total $O(Bd)$.

5.2. Estimación OOB y Grandes Datasets

La estimación *out-of-bag* aprovecha que cada observación queda fuera de aproximadamente $B/3$ árboles. Para la observación i , su predicción OOB es:

$$\hat{f}_{OOB}(\mathbf{x}_i) = \frac{1}{|\mathcal{B}_i^c|} \sum_{b \notin \mathcal{B}_i} f_b(\mathbf{x}_i),$$

donde $\mathcal{B}_i = \{b : (\mathbf{x}_i, y_i) \in \mathcal{D}^{*b}\}$. El error OOB $\frac{1}{n} \sum_i (y_i - \hat{f}_{OOB}(\mathbf{x}_i))^2$ es un estimador insesgado del error de generalización, evitando el costo de la validación cruzada en datasets grandes. Para escenarios de datos masivos, las implementaciones distribuidas (p.ej. `spark-sklearn` o `dask-ml`) replican la lógica de paralelización por árboles entre nodos de un clúster.

6. Implementación en Python

6.1. Uso en scikit-learn

La clase `RandomForestClassifier` (y su análoga `RandomForestRegressor`) de `scikit-learn` implementa fielmente la teoría expuesta. Internamente, cada estimador base es un `DecisionTreeClassifier` con el parámetro `max_features=m`, entrenado sobre una muestra bootstrap de los datos. La agregación se realiza mediante votación mayoritaria (o promedio de probabilidades de clase) en `predict` y `predict_proba`.

Cuadro 1: Hiperparámetros clave

Parámetro	Símbolo	Descripción
<code>n_estimators</code>	B	Núm. de árboles del bosque
<code>max_depth</code>	d	Profundidad máxima de cada árbol
<code>max_features</code>	m	Variables por nodo ($\sqrt{p}$, $p/3, \dots$)
<code>min_samples_split</code>	—	Mínimo de observaciones para dividir un nodo
<code>min_samples_leaf</code>	—	Mínimo de observaciones en una hoja (controla varianza)
<code>bootstrap</code>	—	Activar o desactivar muestreo bootstrap
<code>oob_score</code>	—	Cálculo automático del error OOB
<code>n_jobs</code>	P	Número de núcleos para paralelización
<code>criterion</code>	Q	Medida de impureza: gini o entropy

6.2. Ejemplo

El conjunto de datos sintético generado a continuación incorpora variables informativas, redundantes y ruido, con el objetivo de simular escenarios realistas donde no todas las características contribuyen a la predicción.

Este tipo de configuración permite evaluar la capacidad de modelos como Random Forest para identificar automáticamente las variables relevantes y manejar relaciones no lineales en datos tabulares.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (roc_auc_score,
                             classification_report)
from sklearn.inspection import permutation_importance

# 1000 obs, 20 features, 5 informativas
X, y = make_classification(
    n_samples=1000, n_features=20,
    n_informative=5, n_redundant=5,
    random_state=42)

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y,
    random_state=42)

# B = 500 árboles; m = sqrt(p) (default clasificación)
# max_depth=None => árboles sin restricción (bajo sesgo)
rf = RandomForestClassifier(
    n_estimators=500,      # B: más árboles => menos var
    max_depth=None,       # árboles profundos, bajo sesgo
    max_features="sqrt",  # m = sqrt(p); decorrelación
    min_samples_leaf=1,   # hoja con 1 obs => baja var
    bootstrap=True,       # muestreo bootstrap activado
    oob_score=True,       # estimar error sin CV
    n_jobs=-1,            # usar todos los núcleos (||)
    criterion="gini",      # impureza de Gini
    random_state=42)

rf.fit(X_tr, y_tr)

# Error OOB: estimación insesgada de generalización
print(f"Error OOB: {1 - rf.oob_score_:.4f}")

# Evaluación en test
y_prob = rf.predict_proba(X_te)[: , 1]
auc = roc_auc_score(y_te, y_prob)
print(f"AUC Test : {auc:.4f}")
print(classification_report(y_te, rf.predict(X_te)))

# Importancia por disminución de impureza (MDI)
# = promedio de Delta_Q ponderado por núm de muestras
importances = rf.feature_importances_
top5 = np.argsort(importances)[::-1][5]
for i in top5:
    print(f" Feature {i:2d}: {importances[i]:.4f}")

# Importancia por permutación (para altas dimensiones)
result = permutation_importance(
    rf, X_te, y_te, n_repeats=20,
    random_state=42, n_jobs=-1)
perm_imp = result.importances_mean
```

Cuadro 2: Resultados del modelo de clasificación

Clase	Precisión	Recall	F1-score	Soporte
0	0.91	0.94	0.93	100
1	0.94	0.91	0.92	100
Accuracy		0.93		200
Macro avg	0.93	0.93	0.92	200
Weighted avg	0.93	0.93	0.92	200

Cuadro 3: Importancia de variables

Variable	Importancia
Feature 13	0.2233
Feature 8	0.1258
Feature 11	0.1254
Feature 0	0.0910
Feature 9	0.0673

El modelo presenta un desempeño sólido, con una exactitud global de 93 %, lo que indica una alta capacidad predictiva sobre el conjunto de prueba. Las métricas por clase muestran un comportamiento equilibrado: la clase 0 tiene un recall ligeramente superior (0.94), mientras que la clase 1 presenta una mayor precisión (0.94), lo que sugiere un buen balance entre falsos positivos y falsos negativos.

El valor de AUC de 0.9923 confirma una excelente capacidad de discriminación del modelo, siendo capaz de separar de manera casi perfecta ambas clases. Asimismo, el error Out-of-Bag (OOB) de 0.0687 indica una buena generalización, consistente con el rendimiento observado en el conjunto de prueba.

En cuanto a la importancia de variables, la Feature 13 destaca como la más influyente en las predicciones del modelo, seguida por las Features 8 y 11. Esto sugiere que un subconjunto reducido de variables concentra gran parte del poder explicativo, lo cual es característico de modelos basados en árboles como Random Forest.

6.3. Ajuste de Hiperparámetros

El ajuste de Random Forest es más simple que el de modelos como XGBoost, pues el método es inherentemente robusto. La estrategia recomendada sigue la jerarquía:

1. Fijar B suficientemente grande ($n_estimators \geq 300$) y verificar la convergencia del error OOB;
2. Ajustar $max_features$ entre $\lfloor \sqrt{p} \rfloor$ y $\lfloor p/3 \rfloor$, que controla el tradeoff entre decorrelación de árboles (beneficia varianza) y calidad de cada árbol (beneficia sesgo);
3. Ajustar $min_samples_leaf$ para controlar la profundidad efectiva sin fijar max_depth explícitamente;
4. Habilitar $oob_score=True$ para evaluar sin validación cruzada, especialmente valioso en datasets grandes.

7. Importancia de Variables y Extensiones

7.1. Importancia por Disminución de Impureza (MDI)

Random Forest proporciona de forma natural una medida de importancia de variables denominada *Mean Decrease in Impurity* (MDI) o importancia de Gini. Para la variable k , se define como:

$$\text{MDI}(k) = \frac{1}{B} \sum_{b=1}^B \sum_{t \in T_b: k_t=k} \frac{n_t}{n} \Delta Q(R_t; k, s_t^*),$$

donde la suma interior recorre todos los nodos t del árbol b que usan la variable k para su división, n_t es el número de observaciones en ese nodo y ΔQ es la reducción de impureza correspondiente. La ponderación por n_t/n garantiza que las divisiones cercanas a la raíz (que afectan a más observaciones) tengan mayor peso.

La MDI mide la contribución promedio de la variable k a la reducción total de impureza en el bosque. Sin embargo, presenta un sesgo conocido hacia variables con muchos valores distintos (cardinalidad alta), especialmente variables continuas frente a variables categóricas de baja cardinalidad. Como alternativa más robusta, la **importancia por permutación** mide la degradación del error OOB al permutar aleatoriamente los valores de la variable k :

$$\text{PI}(k) = \frac{1}{|\mathcal{B}_i^c|} \sum_i \left[\ell(y_i, \hat{f}_{OOB}^{\sigma_k}(\mathbf{x}_i)) - \ell(y_i, \hat{f}_{OOB}(\mathbf{x}_i)) \right],$$

donde $\hat{f}_{OOB}^{\sigma_k}$ es la predicción OOB tras permutar la columna k . Una variable irrelevante tiene $\text{PI} \approx 0$; una variable relevante tiene PI positiva, proporcional a la caída en rendimiento.

7.2. Variantes y Extensiones

El marco de Random Forest ha generado diversas extensiones teóricas y prácticas de relevancia. **Extra Trees** (*Extremely Randomized Trees*) lleva la aleatoriedad al extremo, además de seleccionar m variables aleatoriamente, el umbral de cada división también se elige al azar (no mediante optimización), lo que reduce aún más la varianza a costa de un mayor sesgo y produce árboles más decorrelacionados con un costo computacional inferior.

Bosques de Proximidad. Aprovechan la estructura del bosque para definir una métrica de similitud entre observaciones: $d_{RF}(\mathbf{x}, \mathbf{x}') = 1 - f(\mathbf{x}, \mathbf{x}')$, donde $f(\mathbf{x}, \mathbf{x}')$ es la fracción de árboles en los que $\mathbf{x}$ y $\mathbf{x}'$ terminan en la misma hoja. Esta métrica captura relaciones no lineales en el espacio original y se emplea en algoritmos de imputación de datos faltantes (missForest) y en reducción de dimensionalidad no supervisada.

Isolation Forest. Adapta el marco de árboles aleatorios a la detección de anomalías, construye árboles de aislamiento en los que las observaciones atípicas son aisladas más rápidamente (profundidad menor) que las observaciones normales, sin necesidad

de etiquetas supervisadas. Esto ilustra cómo el mismo principio de aleatoriedad y agregación puede trasladarse fuera del paradigma de clasificación y regresión supervisada.

Cuantile Regression Forests. Extienden el estimador promedio de regresión para estimar la distribución condicional completa $F(y | \mathbf{x})$, no solo la media. En lugar de promediar las predicciones de los árboles, se almacenan todas las observaciones de entrenamiento en las hojas activas y se estima el cuantil τ de la distribución empírica en esas hojas, proporcionando intervalos de predicción calibrados.

Redes Neuronales

8. Introducción

Las redes neuronales artificiales constituyen uno de los enfoques más potentes dentro del aprendizaje automático debido a su capacidad para modelar relaciones complejas entre variables. A diferencia de métodos más simples, las redes neuronales aprenden directamente a partir de los datos mediante la construcción progresiva de representaciones internas cada vez más abstractas.

El principio fundamental es la *jerarquía de representaciones*, esto consiste en que cada capa de la red transforma la información recibida en una nueva representación, combinando operaciones lineales con funciones no lineales. Este proceso permite que el modelo capture patrones de creciente complejidad, desde características básicas hasta estructuras altamente sofisticadas.

De manera formal, una red neuronal puede interpretarse como una función compuesta, donde múltiples capas se encadenan entre sí. Cada una de estas capas está parametrizada por pesos y sesgos, los cuales determinan cómo se transforma la información. El objetivo del entrenamiento es encontrar los valores óptimos de estos parámetros para minimizar una función de pérdida $\mathcal{L}(\theta)$ que mide el error del modelo sobre los datos.

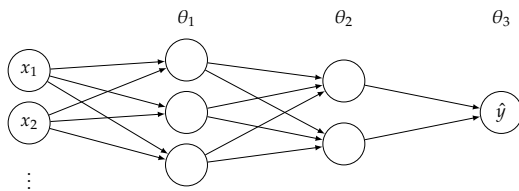


Figura 3: Red neuronal feedforward donde cada capa está parametrizada por θ_l

Este proceso de optimización se lleva a cabo mediante algoritmos de descenso de gradiente, siendo el *backpropagation* el mecanismo central que permite calcular de manera eficiente cómo deben ajustarse los parámetros en cada capa.

Las redes neuronales son especialmente adecuadas para datos no estructurados o con alta complejidad, como imágenes, texto, audio y series de tiempo. Su capacidad para aprender representaciones jerárquicas permite capturar patrones espaciales y temporales que modelos tradicionales no pueden representar fácilmente. Sin embargo, requieren grandes volúmenes de datos y un mayor costo computacional.

9. Fundamentos Teóricos

9.1. La Neurona Artificial

La unidad computacional básica es la **neurona artificial**, definida puramente en términos matemáticos. Dada una entrada $\mathbf{x} \in \mathbb{R}^n$, la neurona calcula:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^n w_j x_j + b, \quad a = \sigma(z),$$

donde $\mathbf{w} \in \mathbb{R}^n$ es el vector de pesos, $b \in \mathbb{R}$ el sesgo, z la *pre-activación* y $a \in \mathbb{R}$ la salida.

La función $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ es la función de activación, su no linealidad es el elemento esencial que permite al modelo aprender representaciones no lineales, pues sin ella la composición de capas sería equivalente a una única transformación lineal.

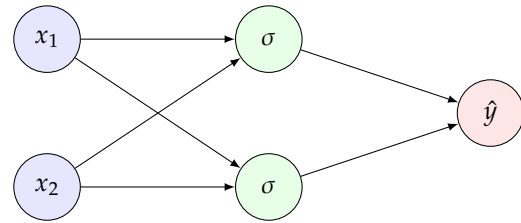


Figura 4: Red neuronal simple donde cada neurona aplica una función de activación $\sigma(\cdot)$.

9.2. Funciones de Activación

Las funciones de activación más relevantes y sus propiedades analíticas son:

Sigmoide. $\sigma(z) = (1 + e^{-z})^{-1}$, imagen en $(0, 1)$, derivada $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Su saturación para $|z| \gg 0$ ($\sigma' \rightarrow 0$) causa el problema del gradiente evanescente en redes profundas.

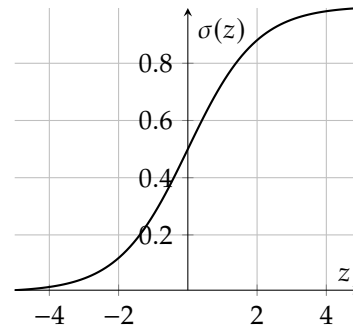


Figura 5: Función sigmoide

Tangente hiperbólica. $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$

$e^{-z})^{-1}$, imagen en $(-1, 1)$, derivada $1 - \tanh^2(z)$. Centrada en cero, mejora la convergencia del gradiente respecto a la sigmoide.

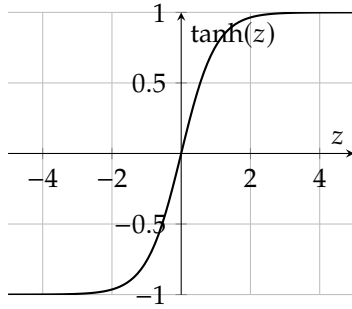


Figura 6: Función tangente hiperbólica

ReLU. $\text{ReLU}(z) = \max(0, z)$, derivada $1[z > 0]$. No satura para $z > 0$, mitigando el gradiente evanescente, y es computacionalmente eficiente. Su limitación principal es la *neurona muerta*: si la pre-activación permanece negativa, la neurona no aprende.

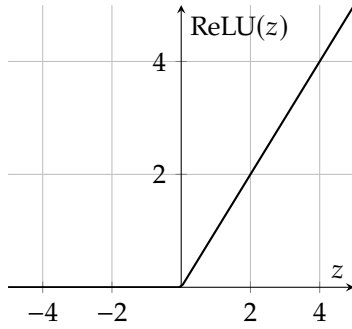


Figura 7: Función ReLU

GELU. $\text{GELU}(z) = z \Phi(z)$, donde Φ es la CDF de la normal estándar. Produce una transición suave y es la activación preferida en modelos de lenguaje de gran escala (Transformers).

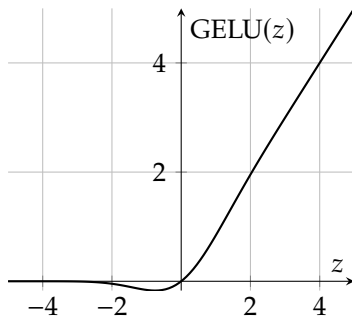


Figura 8: Función GELU (aproximación)

9.3. Red Feedforward como Composición de Funciones

Una **red neuronal feedforward** de L capas ocultas se define como la composición:

$$\begin{aligned} a^{(0)} &= \mathbf{x}, \\ z^{(\ell)} &= W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad \ell = 1, \dots, L, \\ a^{(\ell)} &= \sigma(z^{(\ell)}), \\ f_{\theta}(\mathbf{x}) &= W^{(L+1)} a^{(L)} + b^{(L+1)}, \end{aligned} \quad (9.1)$$

donde $W^{(\ell)} \in \mathbb{R}^{n_{\ell} \times n_{\ell-1}}$, $b^{(\ell)} \in \mathbb{R}^{n_{\ell}}$, y σ se aplica componente a componente (*elementwise*). La dimensión total del espacio de parámetros es:

$$\dim(\theta) = \sum_{\ell=1}^{L+1} (n_{\ell} n_{\ell-1} + n_{\ell}).$$

Intuición 2. Cada capa realiza una transformación afín—rotación, escalado, traslación en $\mathbb{R}^{n_{\ell}}$ —seguida de una no linealidad. Geométricamente, la red “dobla” el espacio de representaciones de forma progresiva: capas profundas pueden implementar fronteras de decisión arbitrariamente complejas al componer muchas transformaciones simples.

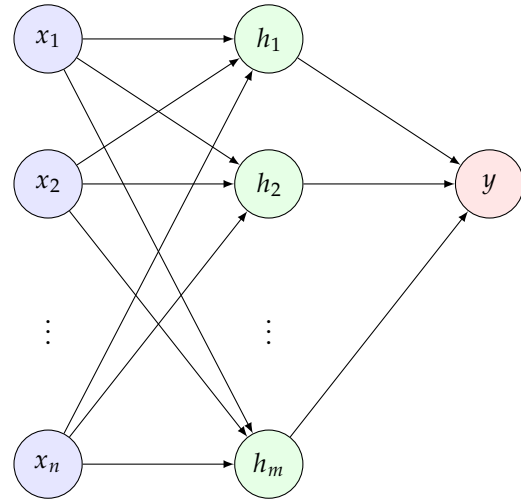


Figura 9: Arquitectura de una red neuronal feedforward con una capa oculta.

9.4. Teorema de Aproximación Universal

Teorema 1 (Hornik, Stinchcombe & White, 1989). Sea $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ continua, acotada y no constante. Para toda función continua $g : \mathcal{K} \rightarrow \mathbb{R}$ sobre un compacto $\mathcal{K} \subset \mathbb{R}^p$ y todo $\varepsilon > 0$, existe $N \in \mathbb{N}$ y parámetros $\{v_j, \mathbf{w}_j, b_j\}_{j=1}^N$ tales que:

$$\sup_{\mathbf{x} \in \mathcal{K}} \left| g(\mathbf{x}) - \sum_{j=1}^N v_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j) \right| < \varepsilon.$$

El resultado garantiza que las redes de una capa oculta son aproximadores universales de funciones continuas. Sin embargo, es un teorema de existencia, es decir, no especifica cómo encontrar los parámetros ni cuántas neuronas se necesitan. En la práctica, las redes profundas son exponencialmente más eficientes que las anchas de una sola capa para representar ciertas familias de funciones, pues la profundidad permite reutilizar representaciones intermedias de forma compositiva, lo que justifica teóricamente el uso del aprendizaje profundo (*Deep Learning*).

10. Construcción Matemática

10.1. Función de Pérdida

El entrenamiento se formula como minimización del riesgo empírico regularizado. Dado $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(\mathbf{y}_i, f_{\theta}(\mathbf{x}_i)) + \lambda \Omega(\theta),$$

donde ℓ es la pérdida por muestra y Ω un término de regularización. Las elecciones más habituales de ℓ son:

Error cuadrático medio (MSE), regresión:

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{n_{L+1}} \|\mathbf{y} - \hat{\mathbf{y}}\|^2.$$

Entropía cruzada binaria, clasificación:

$$\ell(y, \hat{p}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}),$$

donde $\hat{p} = \sigma(f_{\theta}(\mathbf{x})) \in (0, 1)$.

Entropía cruzada categórica, K clases:

$$\ell(\mathbf{y}, \hat{\mathbf{p}}) = - \sum_{k=1}^K y_k \log \hat{p}_k,$$

con $\hat{p}_k = e^{f_k} / \sum_j e^{f_j}$ (softmax). La combinación softmax + entropía cruzada es canónica porque su gradiente tiene forma especialmente limpia: $\nabla_{z^{(L+1)}} \ell = \hat{\mathbf{p}} - \mathbf{y}$, sin el efecto saturador del softmax.

10.2. Derivación Completa de Backpropagation

El algoritmo de *retropropagación* calcula $\nabla_{\theta} \mathcal{L}$ eficientemente aplicando la regla de la cadena de forma sistemática hacia atrás en el grafo computacional de la red. Trabajamos con la pérdida de una sola muestra para claridad de la notación.

Paso 1: Forward pass. Calcular y almacenar $z^{(\ell)}$ y $a^{(\ell)}$ para $\ell = 1, \dots, L+1$ según la recursión (9.1). Este almacenamiento es necesario para el backward pass.

Paso 2: Error en la capa de salida. Definimos el **error** de la última capa como el gradiente de la pérdida respecto a la pre-activación:

$$\delta^{(L+1)} = \frac{\partial \mathcal{L}}{\partial z^{(L+1)}} \in \mathbb{R}^{n_{L+1}}.$$

Para MSE con activación identidad en la salida, $\delta^{(L+1)} = \hat{\mathbf{y}} - \mathbf{y}$. Para entropía cruzada categórica con softmax, $\delta^{(L+1)} = \hat{\mathbf{p}} - \mathbf{y}$.

Paso 3: Regla de retropropagación. Para $\ell = L, L-1, \dots, 1$, el error de la capa ℓ se obtiene aplicando la regla de la cadena en dos pasos:

$$\delta^{(\ell)} = \left(W^{(\ell+1)}\right)^{\top} \delta^{(\ell+1)} \odot \sigma'\left(z^{(\ell)}\right), \quad (10.1)$$

donde $\odot$ denota el producto de Hadamard (componente a componente) y $\sigma'(z^{(\ell)}) \in \mathbb{R}^{n_{\ell}}$ es el vector de derivadas de la activación evaluadas en la pre-activación. La derivación sigue de:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial a^{(\ell-1)}} &= \left(W^{(\ell)}\right)^{\top} \frac{\partial \mathcal{L}}{\partial z^{(\ell)}} = \left(W^{(\ell)}\right)^{\top} \delta^{(\ell)}, \\ \frac{\partial \mathcal{L}}{\partial z^{(\ell)}} &= \frac{\partial \mathcal{L}}{\partial a^{(\ell)}} \odot \sigma'\left(z^{(\ell)}\right) = \delta^{(\ell)}, \end{aligned}$$

donde la segunda línea aplica la regla de la cadena escalar componente a componente sobre la activación $a^{(\ell)} = \sigma(z^{(\ell)})$.

Paso 4: Gradientes respecto a los parámetros. Una vez calculados los errores $\{\delta^{(\ell)}\}_{\ell}$, los gradientes de los pesos y sesgos son:

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(a^{(\ell-1)}\right)^{\top} \in \mathbb{R}^{n_{\ell} \times n_{\ell-1}}, \quad (10.2)$$

$$\frac{\partial \mathcal{L}}{\partial b^{(\ell)}} = \delta^{(\ell)} \in \mathbb{R}^{n_{\ell}}. \quad (10.3)$$

La ecuación (10.2) se obtiene de que $\mathcal{L}$ depende de $W^{(\ell)}$ únicamente a través de $z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$, y $\partial z^{(\ell)} / \partial W^{(\ell)} = a^{(\ell-1)\top}$ en el sentido del producto exterior.

Intuición 3. *Backpropagation no es más que la regla de la cadena aplicada de atrás hacia adelante sobre el grafo computacional. Su eficiencia radica en que cada $\delta^{(\ell)}$ se calcula una sola vez y se reutiliza para los gradientes de todos los parámetros de la capa ℓ . El coste total del backward pass es $O(\dim \theta)$, idéntico al del forward pass, lo que hace computacionalmente tratable el entrenamiento de redes con millones de parámetros.*

10.3. Gradiente Evanesciente y Explosivo

La ecuación (10.1) revela que el error en la capa ℓ involucra el producto iterado de matrices transpuestas y derivadas de activación. En una red de L capas, el gradiente respecto a los parámetros de la primera capa contiene el producto:

$$\delta^{(1)} \propto \prod_{\ell=2}^{L+1} \left(W^{(\ell)}\right)^{\top} \cdot \text{diag}\left(\sigma'\left(z^{(\ell-1)}\right)\right).$$

Si los valores singulares de los pesos o las derivadas de activación son menores que 1, este producto se contrae exponencialmente con L : **gradiente evanescente** (*vanishing gradient*). Recíprocamente, si son mayores que 1, crece exponencialmente: **gradiente explosivo** (*exploding gradient*). Ambos fenómenos dificultan gravemente el entrenamiento de redes profundas, motivando el uso de ReLU/GELU, inicialización cuidadosa, normalización por lotes y conexiones residuales.

11. Optimización

11.1. Descenso por Gradiente y Variantes

El entrenamiento busca resolver:

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^D} \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_\theta(\mathbf{x}_i)) + \lambda \Omega(\theta).$$

El **descenso en lote** actualiza $\theta_{t+1} = \theta_t - \eta_t \nabla_\theta \mathcal{L}(\theta_t)$ con costo $O(n \cdot \dim \theta)$ por iteración. El **descenso estocástico** (SGD) reemplaza el gradiente exacto por un estimador insesgado basado en una sola muestra, con mayor varianza pero efectos regularizadores documentados. El SGD tiende a encontrar mínimos planos que generalizan mejor que los mínimos agudos hallados por GD. El mini-batch SGD, que evalúa el gradiente en un subconjunto $\mathcal{B}_t$ de tamaño $B \ll n$:

$$\hat{g}_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_\theta \ell(y_i, f_\theta(\mathbf{x}_i)),$$

equilibra la eficiencia computacional con una varianza moderada y es el estándar en la práctica.

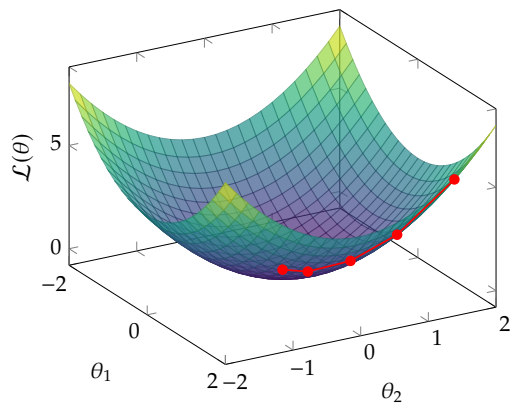


Figura 10: Descenso por gradiente sobre una función de pérdida $\mathcal{L}(\theta)$ en un espacio de parámetros bidimensional.

11.2. Momentum y Adam

El **momentum** acumula una media exponencialmente ponderada de los gradientes pasados para suavizar la trayectoria:

$$v_{t+1} = \mu v_t - \eta_t \hat{g}_t, \quad \theta_{t+1} = \theta_t + v_{t+1},$$

con $\mu \in [0, 1)$ (típicamente 0.9). El vector de velocidad v_t acumula inercia en las direcciones de gradiente consistentes y cancela oscilaciones perpendiculares, acelerando la convergencia en valles estrechos de la superficie de pérdida.

Adam, el optimizador más utilizado en aprendizaje profundo, combina momentum de primer y segundo orden para producir una tasa de aprendizaje adaptativa por parámetro:

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \hat{g}_t, \quad (11.1)$$

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) \hat{g}_t^2, \quad (11.2)$$

$$\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}, \quad (11.3)$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1} + \epsilon}}. \quad (11.4)$$

Las ecuaciones (11.1)–(11.2) mantienen medias del gradiente y de su cuadrado. La corrección (11.3) compensa la inicialización en cero de los momentos. La actualización (11.4) normaliza el paso por la desviación estándar histórica del gradiente: parámetros con gradientes pequeños reciben pasos más grandes. Los valores por defecto $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$ son robustos en la mayoría de aplicaciones.

12. Supuestos y Propiedades

12.1. Capacidad y Descomposición Sesgo-Varianza

La descomposición clásica del error de generalización es:

$$\mathbb{E}[(y - f_\theta(\mathbf{x}))^2] = \text{Bias}^2[f_\theta] + \text{Var}[f_\theta] + \sigma_\epsilon^2.$$

Redes con pocos parámetros tienen sesgo alto y varianza baja (subajuste); redes con muchos parámetros tienen sesgo bajo pero pueden sobreajustar (alta varianza). Un fenómeno peculiar del aprendizaje profundo es la doble curva de descenso (*double descent*): en el régimen sobreparametrizado ($\dim \theta \gg n$), el error de generalización *desciende nuevamente* conforme crece la capacidad, siempre que el entrenamiento se realice con SGD e inicialización adecuada. Este fenómeno contradice la intuición clásica del tradeoff sesgo-varianza y tiene su origen en el sesgo implícito de SGD hacia soluciones de norma mínima en el espacio sobreparametrizado.

12.2. Regularización

Regularización L_2 (weight decay). Añade la penalización $\lambda \sum_\ell \|W^{(\ell)}\|_F^2$ a la pérdida, equivalente a un a priori gaussiano sobre los pesos en la interpretación bayesiana. Produce una contracción uniforme de todos los pesos hacia el origen.

Regularización L_1 . $\lambda \sum_\ell \|W^{(\ell)}\|_1$, equivalente a un a priori de Laplace, induce sparsidad en los pesos

porque su subgradiente es constante ($\pm\lambda$) independientemente de la magnitud del peso.

Dropout. Durante el entrenamiento, cada neurona se desactiva con probabilidad p en cada forward pass:

$$\tilde{a}_j^{(\ell)} = \frac{m_j}{1-p} a_j^{(\ell)}, \quad m_j \sim \text{Bernoulli}(1-p).$$

La reescalación por $1/(1-p)$ garantiza que la esperanza de la activación sea igual a la original en inferencia (donde $m_j \equiv 1$). Dropout puede interpretarse como entrenamiento implícito de un conjunto de 2^N redes con parámetros compartidos, con efecto regularizador similar al bagging pero con costo computacional de un solo modelo.

Batch Normalization. Normaliza las activaciones dentro de cada mini-batch para estabilizar la distribución de entradas a cada capa:

$$\hat{z}_j^{(\ell)} = \frac{z_j^{(\ell)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad \tilde{z}_j^{(\ell)} = \gamma_j \hat{z}_j^{(\ell)} + \beta_j,$$

con γ_j, β_j aprendibles. Batch Norm reduce la dependencia de la inicialización, permite tasas de aprendizaje más altas y actúa como regularizador implícito al introducir ruido en las estimaciones de μ y σ^2 derivado del mini-batch.

13. Aspectos Computacionales

13.1. Complejidad y Paralelismo en GPU

El costo de un forward pass sobre un mini-batch de tamaño B es:

$$C_{\text{fwd}} = 2B \sum_{\ell=1}^{L+1} n_{\ell} \cdot n_{\ell-1} \approx 2B \cdot n_{\max}^2 \cdot L,$$

donde $n_{\max} = \max_{\ell} n_{\ell}$. El backward pass tiene costo numéricamente equivalente, requiriendo además el almacenamiento de las activaciones del forward. El costo total de una época con n muestras es $O(n \cdot n_{\max}^2 \cdot L)$.

La operación fundamental es la multiplicación de matrices, que se paraleliza de forma óptima en GPU. El mini-batch SGD explota este paralelismo al procesar B muestras como una única operación matricial:

$$Z^{(\ell)} = W^{(\ell)} A^{(\ell-1)} + b^{(\ell)} \mathbf{1}^T, \quad A^{(\ell-1)} \in \mathbb{R}^{n_{\ell-1} \times B}.$$

Una GPU moderna dispone de miles de núcleos con ancho de banda de memoria ~ 1 TB/s, dos órdenes de magnitud superior al de una CPU, lo que permite acelerar el entrenamiento típicamente entre $10\times$ y $1000\times$ según la arquitectura. Para modelos de gran escala (miles de millones de parámetros), se

emplean técnicas de paralelismo de modelo (*tensor parallelism*, *pipeline parallelism*) que distribuyen los tensores de pesos entre múltiples aceleradores, y paralelismo de datos que distribuye los mini-batches.

13.2. Inicialización de Pesos

La inicialización de pesos es crítica para evitar el gradiente evanescente desde la primera iteración. La inicialización elige los pesos de modo que la varianza de las pre-activaciones sea constante a lo largo de las capas para activaciones simétricas (tanh):

$$W_{ij}^{(\ell)} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\ell-1} + n_{\ell}}}, +\sqrt{\frac{6}{n_{\ell-1} + n_{\ell}}}\right).$$

La inicialización de He (2015), diseñada para ReLU, compensa que ReLU elimina la mitad de las activaciones en esperanza:

$$W_{ij}^{(\ell)} \sim \mathcal{N}\left(0, \frac{2}{n_{\ell-1}}\right).$$

14. Implementación en Python

14.1. Uso en PyTorch

PyTorch implementa redes neuronales mediante grafos computacionales dinámicos, esto consiste en que cada operación construye automáticamente el grafo necesario para el backward pass, calculando los gradientes por diferenciación automática (*autograd*). Al invocar `loss.backward()`, el motor recorre el grafo de atrás hacia adelante acumulando los gradientes en el atributo `.grad` de cada tensor de parámetros, aplicando exactamente las ecuaciones (10.1)–(10.3).

Cuadro 4: Hiperparámetros clave

Parámetro	Símb.	Rol teórico
<code>learning_rate</code>	η	Tamaño del paso GD
<code>epochs</code>	T	Iteraciones sobre $\mathcal{D}$
<code>batch_size</code>	B	Tamaño del mini-batch
<code>hidden_size</code>	n_{ℓ}	Anchura de capa oculta
<code>num_layers</code>	L	Profundidad de la red
<code>dropout</code>	p	Prob. de desactivación
<code>weight_decay</code>	λ	Coef. regularización L_2
<code>beta1</code> , <code>beta2</code>	β_1, β_2	Momentos de Adam
<code>activation</code>	σ	Función de activación

14.2. Ejemplo

A continuación se implementa una red neuronal feedforward para un problema de clasificación binaria usando datos sintéticos. Los datos se normalizan y se dividen en entrenamiento y prueba.

El modelo se entrena con entropía cruzada binaria y el optimizador Adam, incorporando regularización y ajuste dinámico de la tasa de aprendizaje. Su desempeño se evalúa mediante el AUC.

Dado que los datos fueron generados mediante un proceso de clasificación binaria con un subconjunto de variables informativas, se espera que el modelo aprenda a identificar patrones en dichas variables para distinguir entre las dos clases. En particular, el modelo debe aproximar la probabilidad de pertenencia a cada clase y asignar valores cercanos a 0 o 1 según corresponda, maximizando así su capacidad de discriminación.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score

# 1 Datos
# Clasificación binaria: 2000 muestras, 20 features
X, y = make_classification(
    n_samples=2000, n_features=20,
    n_informative=10, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Normalización: media=0, varianza=1 por columna
sc = StandardScaler()
X_tr = sc.fit_transform(X_tr)
X_te = sc.transform(X_te)

Xtr = torch.FloatTensor(X_tr)
ytr = torch.FloatTensor(y_tr).unsqueeze(1)
Xte = torch.FloatTensor(X_te)
yte = torch.FloatTensor(y_te).unsqueeze(1)

# Mini-batch: B=64, permutación aleatoria por época
loader = DataLoader(
    TensorDataset(Xtr, ytr),
    batch_size=64, shuffle=True)

# 2 Arquitectura
#  $f(x) = W^{(4)} \sigma(W^{(3)} \sigma(W^{(2)} x + b^{(1)})) + b^{(2)} + b^{(3)} + b^{(4)}$ 
class RedFF(nn.Module):
    """
    Red feedforward de L=3 capas ocultas.
    Arquitectura: 20->128->64->32->1
    Activación: ReLU (mitiga gradiente evanescente)
    Regularización: BatchNorm + Dropout(p=0.3)
    """
    def __init__(self, n_in=20, p=0.3):
        super().__init__()
        self.net = nn.Sequential(
            # Capa 1:  $W^{(1)}$  en  $R^{128 \times 20}$ 
            nn.Linear(n_in, 128),
            nn.BatchNorm1d(128), # normaliz. batch
            nn.ReLU(),
            nn.Dropout(p), # Dropout: p=0.3

            # Capa 2:  $W^{(2)}$  en  $R^{64 \times 128}$ 
            nn.Linear(128, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(p),

            # Capa 3:  $W^{(3)}$  en  $R^{32 \times 64}$ 
            nn.Linear(64, 32),
            nn.ReLU(),
```

```
            # Capa salida:  $W^{(4)}$  en  $R^{1 \times 32}$ 
            nn.Linear(32, 1),
            nn.Sigmoid() # salida en (0,1)
        )

    def forward(self, x):
        return self.net(x)

model = RedFF()

# Inicialización He para capas con ReLU
# Garantiza:  $\text{Var}(z^{(l)}) = \text{Var}(z^{(l-1)})$ 
for m in model.modules():
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(
            m.weight, nonlinearity='relu')
        nn.init.zeros_(m.bias)

# 3 Pérdida y optimizador
# BCE:  $-y \log p - (1-y) \log(1-p)$ 
criterion = nn.BCELoss()

# Adam: ecs. (4)-(7); weight_decay = lambda L2
optimizer = optim.Adam(
    model.parameters(),
    lr=1e-3, # eta = 0.001
    betas=(0.9, 0.999), # beta1, beta2
    weight_decay=1e-4) # lambda = 1e-4

# Reduce eta si val_loss no mejora en 10 épocas
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, patience=10, factor=0.5)

# 4 Loop de entrenamiento
for epoch in range(100):
    model.train() # activa Dropout y BatchNorm
    ep_loss = 0.0
    for xb, yb in loader:
        optimizer.zero_grad() # limpia gradientes
        pred = model(xb) # forward pass ec.(1)
        loss = criterion(pred, yb)
        loss.backward() # backprop ecs.(3)-(6)
        # Gradient clipping: mitiga grad. explosivo
        nn.utils.clip_grad_norm_(
            model.parameters(), max_norm=1.0)
        optimizer.step() # actualización Adam
        ep_loss += loss.item()

    # Validación sin cálculo de gradientes
    model.eval()
    with torch.no_grad():
        vl = criterion(model(Xte), yte)
    scheduler.step(vl)

    if (epoch + 1) % 25 == 0:
        avg = ep_loss / len(loader)
        print(f"Ep {epoch+1:3d} | "
              f"train={avg:.4f} | val={vl:.4f}")

# 5 Evaluación
model.eval()
with torch.no_grad():
    probs = model(Xte).numpy().flatten()
auc = roc_auc_score(y_te, probs)
print(f"\nAUC Test: {auc:.4f}")
```

Cuadro 5: Evolución del entrenamiento del modelo

Época	Loss entrenamiento	Loss validación
25	0.1919	0.2048
50	0.1435	0.1871
75	0.1086	0.1854
100	0.1122	0.1855

El modelo presenta una disminución progresiva en la función de pérdida durante el entrenamiento, lo

que indica un aprendizaje adecuado de los datos. La pérdida de validación también disminuye y posteriormente se estabiliza, sugiriendo una buena capacidad de generalización sin evidencia clara de sobreajuste. Asimismo, el valor de $AUC = 0.9815$ refleja un excelente desempeño del modelo, con alta capacidad para discriminar entre las clases.

14.3. Ajuste de Hiperparámetros

El ajuste de hiperparámetros en redes neuronales sigue una lógica jerárquica, donde algunos parámetros tienen un impacto mucho más decisivo que otros. En particular, la tasa de aprendizaje η es el elemento más crítico: si es demasiado alta, el entrenamiento puede volverse inestable y oscilar sin converger; si es demasiado baja, el proceso se vuelve excesivamente lento. Una estrategia práctica para calibrarla es el *learning rate range test*, que consiste en incrementarla gradualmente durante un entrenamiento corto y elegir el valor en el que la pérdida comienza a disminuir con mayor rapidez.

Una vez fijada la tasa de aprendizaje, la atención se centra en la arquitectura del modelo —es decir, su profundidad L , la anchura de las capas n_l y el tipo de función de activación—, ya que estos elementos determinan la capacidad de representación. En general, modelos más grandes pueden capturar patrones más complejos, pero también requieren mayor cantidad de datos y mecanismos de regularización para evitar el sobreajuste.

Por su parte, el tamaño del mini-batch B tiene un efecto más sutil, aunque relevante. En la práctica, tamaños pequeños (por ejemplo, $B \in [16, 128]$) suelen favorecer la generalización debido al ruido inherente en la estimación del gradiente, mientras que tamaños grandes permiten aprovechar mejor el paralelismo computacional, especialmente en GPU.

La diferencia entre la pérdida de entrenamiento y la de validación proporciona una guía útil para ajustar técnicas de regularización como *dropout* o *weight decay*. Una brecha amplia entre ambas suele ser indicio de sobreajuste y sugiere la necesidad de incrementar la regularización.

15. Arquitecturas Canónicas

15.1. Redes Convolucionales

Las **redes neuronales convolucionales** (CNN, LeCun *et al.*, 1998) explotan la estructura espacial de los datos mediante la operación de **convolución discreta**. Para una imagen $X \in \mathbb{R}^{H \times W}$ y un filtro $K \in \mathbb{R}^{h \times w}$, el mapa de características resultante es:

$$(X * K)[i, j] = \sum_{m=0}^{h-1} \sum_{n=0}^{w-1} K[m, n] X[i + m, j + n].$$

El filtro K es el parámetro aprendible. La convolución impone dos sesgos inductivos cruciales: *invarianza traslacional* (el mismo filtro se aplica en cada posición espacial) y *localidad* (cada neurona del mapa de salida sólo depende de una pequeña región de la entrada). Estos sesgos reducen drásticamente el número de parámetros respecto a una capa densa: un filtro 3×3 tiene sólo 9 parámetros independientemente del tamaño de la imagen. En la práctica, una capa convolucional aplica C filtros simultáneamente, produciendo un tensor $\mathcal{F} \in \mathbb{R}^{C \times H' \times W'}$, seguido de una no linealidad y una operación de *pooling* que reduce la resolución espacial y aumenta la invarianza local.

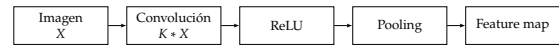


Figura 11: Arquitectura básica de una red convolucional: convolución, activación y pooling.

15.2. Redes Recurrentes y LSTM

Las **redes recurrentes** (RNN) procesan secuencias de longitud variable manteniendo un estado oculto $h_t \in \mathbb{R}^d$ actualizado en cada paso:

$$h_t = \sigma(W_{hh}h_{t-1} + W_{xh}x_t + b_h), \quad \hat{y}_t = W_{hy}h_t + b_y.$$

El gradiente respecto a h_0 involucra el producto de T matrices W_{hh} , que se contrae o diverge exponencialmente con T : gradiente evanescente en el tiempo. Las **Long Short-Term Memory** (LSTM) resuelven este problema mediante compuertas diferenciiables:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad (\text{olvido})$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad (\text{entrada})$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c),$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (\text{celda})$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(c_t).$$

El estado de celda c_t proporciona un camino de gradiente sin atenuación a través de muchos pasos, resolviendo el gradiente evanescente temporal y permitiendo memorizar dependencias a largo plazo.

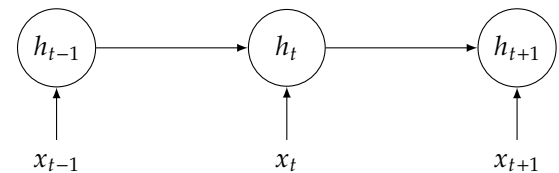


Figura 12: Red neuronal recurrente (RNN) desenrollada en el tiempo.

eXtreme Gradient Boosting

16. Introducción

El algoritmo *eXtreme Gradient Boosting* (XGBoost) representa una evolución sofisticada de los métodos de ensamble basados en árboles de decisión, diseñada específicamente para optimizar la velocidad y el rendimiento computacional. A diferencia de *Random Forest*, que construye múltiples árboles de forma independiente mediante *bagging*, XGBoost implementa el paradigma de *boosting* secuencial. En este enfoque, los modelos se entrenan de manera sucesiva para corregir los errores residuales de las iteraciones previas, utilizando un algoritmo de descenso de gradiente.

En el marco de la Administración Integral de Riesgos, XGBoost se ha consolidado como una herramienta fundamental debido a su capacidad para manejar datos dispersos y su flexibilidad para optimizar diversas funciones de pérdida. Su arquitectura integra técnicas avanzadas de regularización que controlan la complejidad del modelo, lo que resulta crítico para la estabilidad de las predicciones en carteras de crédito y cobranza, donde evitar el sobreajuste es vital para la solvencia institucional.

17. Fundamentos Teóricos

El sustento técnico de XGBoost, basado fundamentalmente en el trabajo de Chen y Guestrin (2016), se apoya en el concepto de aprendizaje aditivo y la optimización en espacios funcionales. A diferencia de los modelos paramétricos tradicionales, este algoritmo no ajusta coeficientes globales, sino que construye una función predictiva mediante la suma secuencial de modelos base.

17.1. Aprendizaje Estadístico Aditivo

El modelo se construye de forma iterativa. Si definimos $\hat{y}_i^{(t)}$ como la predicción para la instancia i en la iteración t , el sistema añade un nuevo árbol f_t que busca minimizar el error remanente del paso anterior:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

Donde f_t es el nuevo aprendiz (*weak learner*) que se incorpora al ensamble para mejorar la capacidad predictiva global mediante la corrección sistemática de los residuos de los modelos precedentes.

17.2. Optimización de la Función Objetivo

Una de las innovaciones que distingue a XGBoost es la inclusión de un término de regularización en la función objetivo $\mathcal{L}(\phi)$. La estructura de esta función busca un equilibrio entre la fidelidad a los datos y la simplicidad estructural del modelo:

$$\mathcal{L}(\phi) = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

Aquí, L representa la función de pérdida (como *log-loss* para calificación de riesgo de crédito), mientras que Ω es el término de penalización que controla el número de hojas y la magnitud de los pesos en los nodos terminales.

18. Construcción matemática

A diferencia de los métodos de *Gradient Boosting* tradicionales que realizan un descenso de gradiente de primer orden, XGBoost utiliza una aproximación de segundo orden para optimizar la función objetivo de manera más eficiente y precisa.

18.1. Expansión de Taylor de la Función de Pérdida

Dada la función objetivo en la iteración t :

$$\mathcal{L}^{(t)} = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Para optimizar esta función para cualquier L diferenciable, aplicamos una expansión de Taylor de segundo grado alrededor del valor previo $\hat{y}_i^{(t-1)}$. Definimos los gradientes de primer y segundo orden como:

$$g_i = \partial_{\hat{y}_i^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)}), \quad h_i = \partial_{\hat{y}_i^{(t-1)}}^2 L(y_i, \hat{y}_i^{(t-1)})$$

La aproximación de la función objetivo, eliminando los términos constantes que no dependen de f_t , se reduce a:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$$

18.2. Determinación de Pesos Óptimos

Si definimos la estructura del árbol como una función que asigna cada muestra a una hoja $j \in \{1, \dots, T\}$, con un peso asociado w_j , y definimos la regularización como $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$,

podemos agrupar las instancias por hoja. Para una estructura de árbol fija, el peso óptimo w_j^* de la hoja j que minimiza la pérdida es:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

Sustituyendo este valor, obtenemos la función de estructura (*scoring function*) que mide la calidad de un árbol:

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

18.3. Algoritmo de División (Gain)

En la práctica, es imposible evaluar todas las estructuras de árbol posibles. XGBoost utiliza un algoritmo codicioso (*greedy*) que evalúa la ganancia de realizar una partición en un nodo. La ganancia se calcula como:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

Donde G y H son las sumas de los gradientes y hessianos de los subconjuntos izquierdo (L) y derecho (R). Si la ganancia es menor que γ , la división no se realiza, lo que funciona como una poda automática (*pruning*).

19. Supuestos y Propiedades

A diferencia de los modelos estadísticos clásicos, XGBoost es un modelo no paramétrico que no requiere supuestos estrictos sobre la distribución de los datos (como normalidad o homocedasticidad). No obstante, su efectividad se basa en ciertas propiedades matemáticas y operativas:

19.1. Diferenciabilidad de la Función de Pérdida

El supuesto matemático más crítico es que la función de pérdida $L(y, \hat{y})$ debe ser dos veces diferenciable de manera convexa. Esto es necesario para poder calcular el gradiente (g_i) y el hessiano (h_i) requeridos en la expansión de Taylor. Esta propiedad permite que XGBoost sea extremadamente flexible, pudiendo utilizarse para regresión, clasificación y modelos de supervivencia.

19.2. Tratamiento de Valores Faltantes (Sparsity Awareness)

Una propiedad distintiva de XGBoost es su capacidad nativa para manejar datos dispersos o valores nulos. El algoritmo aprende una "dirección por defecto" en cada nodo de división. Si un dato tiene un

valor faltante, se asigna automáticamente a la rama que minimiza la pérdida durante el entrenamiento. Esto elimina la necesidad de imputación manual de datos, un supuesto común en otros modelos de *Machine Learning*.

19.3. Control de Sobreajuste mediante Regularización

A diferencia del *Gradient Boosting* convencional, XGBoost incluye regularización L_1 (Lasso) y L_2 (Ridge) en su construcción. Esta propiedad garantiza que el modelo penalice la complejidad excesiva (hojas con pesos muy altos), asegurando que el estimador sea consistente y mantenga un equilibrio óptimo en la descomposición sesgo-varianza, fundamental en la administración de riesgos para evitar modelos sobreajustados a datos históricos.

19.4. Invarianza a la Escala

Al basarse en particiones binarias de variables, el modelo es invariante a la escala de las características de entrada. No se asume que las variables deban estar estandarizadas o normalizadas, lo que permite trabajar directamente con variables financieras de distintas magnitudes (ej. ingresos vs. edad).

20. Aspectos computacionales

XGBoost se distingue de otras implementaciones de *gradient boosting* por su enfoque en la eficiencia del sistema y la escalabilidad. Las innovaciones en su arquitectura computacional permiten el entrenamiento de modelos complejos en tiempos reducidos, incluso con conjuntos de datos masivos.

20.1. Aprendizaje en Paralelo y Estructura de Bloques

Aunque el proceso de *boosting* es intrínsecamente secuencial, XGBoost paraleliza la construcción de los nodos individuales. Mediante una estructura de datos denominada *block structure*, los datos se ordenan y almacenan en memoria, permitiendo que el algoritmo busque los puntos de división (*split points*) de múltiples variables de forma simultánea, optimizando el uso de los núcleos de procesamiento de la CPU.

20.2. Algoritmo de Cuantiles Aproximados

Para conjuntos de datos que exceden la capacidad de la memoria RAM, el algoritmo implementa un *Weighted Quantile Sketch*. En lugar de realizar una búsqueda exacta de todos los posibles puntos de división, XGBoost propone candidatos basados en la distribución de los datos. Esta técnica reduce la complejidad computacional y el uso de memoria, manteniendo una precisión comparable al método de búsqueda exacta.

20.3. Optimización de Caché y Computación Out-of-Core

El algoritmo maximiza el rendimiento del hardware mediante el uso consciente de la memoria caché (*Cache-aware Access*), almacenando las estadísticas de los gradientes de manera que se minimicen los fallos de lectura en el procesador. Asimismo, incorpora capacidades *out-of-core* que permiten procesar datos directamente desde el almacenamiento secundario (disco duro) mediante técnicas de compresión y fragmentación de bloques (*block sharding*), facilitando el análisis de carteras de crédito de gran escala que no podrían procesarse con métodos tradicionales.

21. Interpretación y Atribución de Variables

A pesar de ser un modelo complejo basado en ensambles, XGBoost proporciona herramientas robustas para la interpretabilidad, lo cual es un requisito indispensable en la gestión de riesgos para justificar las decisiones de crédito y cobranza ante los órganos reguladores.

21.1. Puntuación de Importancia (F-score)

XGBoost permite cuantificar la relevancia de cada variable predictora a través de tres métricas fundamentales:

- **Peso (Weight):** El número de veces que una variable se utiliza para dividir los datos en todos los árboles del ensamble.
- **Ganancia (Gain):** La mejora relativa en la precisión que aporta una variable. Es la métrica más importante para entender qué factores (ej. nivel de ingresos o historial crediticio) reducen más la incertidumbre del modelo.
- **Cobertura (Cover):** El número relativo de observaciones relacionadas con una variable específica.

21.2. Valores SHAP y Explicabilidad Local

Para una administración de riesgos más sofisticada, XGBoost es compatible con la teoría de juegos cooperativos a través de los valores SHAP (*SHapley Additive exPlanations*). A diferencia de la importancia global, los valores SHAP permiten descomponer la predicción de un solo cliente, identificando exactamente qué variables aumentaron o disminuyeron su probabilidad de impago individual. Esta capacidad de "explicabilidad local" permite a los gestores de cobranza diseñar estrategias personalizadas basadas en el comportamiento específico de cada deudor.

22. Control de Sobreajuste mediante Parada Temprana

Una propiedad operativa crítica en XGBoost es el *Early Stopping*. Dado que el aprendizaje es aditivo y secuencial, existe el riesgo de que el modelo comience a memorizar el ruido de los datos históricos de entrenamiento.

Mediante el monitoreo de una métrica de validación externa (como el área bajo la curva ROC o la pérdida logarítmica), el proceso de construcción de árboles se detiene automáticamente cuando el error en los datos no observados deja de disminuir. Esta técnica garantiza que el modelo final posea la máxima capacidad de generalización, asegurando su eficacia en escenarios económicos cambiantes donde el comportamiento de pago de los clientes puede divergir de los patrones históricos.

23. Implementación en Python

La implementación de XGBoost en Python se realiza principalmente a través de la biblioteca `xgboost`, la cual ofrece una interfaz optimizada y compatible con `scikit-learn`. En el ámbito de la administración de riesgos, este modelo es ampliamente utilizado para tareas de clasificación binaria, como la determinación de la probabilidad de impago (*default*).

A continuación, se presenta un ejemplo de implementación utilizando un conjunto de datos sintético que simula indicadores de riesgo financiero:

```
import xgboost as xgb
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score,
classification_report
from sklearn.datasets import make_classification

# 1. Generacion de datos (Simulacion de variables de
# riesgo)
X, y = make_classification(n_samples=1000, n_features=20,
                          n_informative=15, n_redundant=5,
                          random_state=42)

# 2. Division en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2,
                                                    random_state=42)

# 3. Definicion del modelo con parametros de regularizacion
model = XGBClassifier(
    n_estimators=100,          # Numero de arboles
    learning_rate=0.1,         # Eta (tasa de aprendizaje)
    max_depth=6,              # Profundidad maxima
    objective='binary:logistic', # Funcion de perdida
    reg_lambda=1,              # Regularizacion L2 (Ridge)
    reg_alpha=0.1,             # Regularizacion L1 (Lasso)
    gamma=0.5,                 # Penalizacion por nuevas hojas
    eval_metric='logloss',
    random_state=42
)

# 4. Entrenamiento del modelo
```

```
model.fit(X_train, y_train)

# 5. Evaluacion del desempeno en cobranza
y_prob = model.predict_proba(X_test)[: , 1]
y_pred = model.predict(X_test)

print(f"AUC-ROC Score: {roc_auc_score(y_test,
                                       y_prob):.4f}")
print(classification_report(y_test, y_pred))
```

23.1. Configuración de hiperparámetros

En la implementación presentada, la selección de hiperparámetros es crucial para garantizar la estabilidad del modelo en un entorno de riesgo:

- **learning_rate (η):** Al establecerse en 0.1, se busca que el aprendizaje sea gradual, evitando que el modelo se ajuste demasiado rápido a anomalías temporales en los datos de cobranza.
- **reg_lambda y reg_alpha:** Estos parámetros controlan la complejidad de los pesos de las hojas, asegurando que el modelo mantenga una estructura simplificada y generalizable ante nuevos clientes.
- **gamma (γ):** Representa el umbral de ganancia mínima necesario para realizar una partición. Esto implementa la "poda" automática que previene la creación de ramas que no aportan una reducción significativa de la pérdida.

Análisis Comparativo de Modelos

24. Síntesis Metodológica

Tras haber analizado de manera individual la construcción y el funcionamiento de Random Forest, XGBoost y las Redes Neuronales, es fundamental establecer una comparativa directa que permita identificar el modelo óptimo según la naturaleza de los datos y los objetivos de la gestión de riesgos.

La elección de un algoritmo sobre otro no depende únicamente de su precisión métrica, sino del equilibrio entre su capacidad predictiva, su interpretabilidad ante reguladores y su robustez operativa en entornos de producción.

Cuadro 6: Cuadro comparativo de modelos de Machine Learning en el contexto de riesgos.

Criterio de comparación	Random Forest	XGBoost	Redes neuronales
Fundamento metodológico	Método de ensamble basado en la construcción de múltiples árboles de decisión generados de manera independiente mediante muestras bootstrap y selección aleatoria de variables en cada nodo.	Método de ensamble secuencial basado en boosting, en el que cada nuevo árbol se ajusta para corregir los errores cometidos por el conjunto de árboles previamente construido.	Modelo de aprendizaje basado en capas sucesivas de unidades neuronales artificiales, cuyo propósito es aproximar funciones complejas mediante composiciones de transformaciones lineales y no lineales.
Capacidad de modelación	Permite capturar relaciones no lineales e interacciones entre variables explicativas sin requerir una especificación funcional previa.	Presenta una elevada capacidad para modelar relaciones no lineales e interacciones complejas, con un ajuste más refinado que otros métodos de árboles agregados.	Posee una alta flexibilidad funcional, lo que le permite aproximar relaciones altamente complejas, profundas y de difícil representación mediante modelos tradicionales.
Fortalezas principales	Se caracteriza por su robustez, estabilidad y buen desempeño en datos tabulares. Tolera adecuadamente ruido, variables irrelevantes y estructuras no lineales. Además, constituye una alternativa metodológicamente sólida como modelo base o de referencia.	Destaca por su elevado poder predictivo en problemas tabulares estructurados, así como por la incorporación explícita de regularización y por su eficiencia computacional en implementaciones modernas. Resulta particularmente eficaz en tareas de clasificación, scoring y ranking.	Sobresale por su flexibilidad y por su capacidad para aprender representaciones complejas a partir de grandes volúmenes de datos. Su utilidad es mayor en contextos donde la información presenta estructuras profundas, secuenciales o no estructuradas.
Debilidades principales	Aunque reduce la varianza frente a un árbol individual, su interpretación es menos directa que la de modelos más simples. Asimismo, en algunos contextos puede ofrecer un desempeño inferior al de métodos boosting.	Su desempeño depende en mayor medida de una correcta calibración de hiperparámetros. Si no se controla adecuadamente la complejidad del modelo, puede presentar sobreajuste. Su interpretabilidad también es más limitada que la de modelos lineales o árboles individuales.	Requiere muestras de mayor tamaño, mayor capacidad computacional y un diseño más cuidadoso del proceso de entrenamiento. Asimismo, su interpretación suele ser reducida, lo que limita su transparencia en algunos contextos aplicados.
Nivel de interpretabilidad	Intermedio. Permite obtener medidas de importancia de variables y cierto nivel de interpretación global, aunque no ofrece reglas tan claras como un árbol individual.	Intermedio a bajo. Si bien existen herramientas auxiliares de explicación, su estructura secuencial dificulta una interpretación directa.	Bajo. En la mayoría de los casos se considera un modelo de baja trazabilidad analítica, debido a la complejidad interna de sus parámetros y representaciones.
Desempeño esperado en datos tabulares	Generalmente alto, especialmente en conjuntos de datos estructurados con múltiples interacciones.	Muy alto, siendo frecuentemente superior a otros métodos en problemas tabulares de clasificación y predicción.	Variable. En datos tabulares tradicionales no siempre supera a modelos basados en árboles, salvo que el volumen o complejidad de los datos lo justifique.
Riesgo de sobreajuste	Relativamente contenido por la agregación de árboles.	Potencialmente elevado si no se regula adecuadamente la profundidad, el número de iteraciones o la tasa de aprendizaje.	Elevado si no se implementan mecanismos de regularización apropiados, tales como penalización, abandono aleatorio o detención temprana.

Ámbito de aplicación más favorable	Problemas tabulares en los que se requiera un equilibrio entre solidez predictiva, robustez y relativa facilidad de implementación.	Problemas tabulares en los que el objetivo principal sea maximizar precisión predictiva o construir esquemas de priorización y scoring.	Problemas complejos con información masiva, secuencial, textual o multi-modal, donde la estructura de los datos justifique una arquitectura de mayor profundidad.
---	---	---	---

25. Referencias

Bishop, C. M., & Bishop, H. (2023). *Deep learning: Foundations and concepts*. Springer.

Gustavo-Aircraft. (s.f.). *WideTrainingExample_Puebla.ipynb* [Archivo de código]. GitHub. https://github.com/Gustavo-Aircraft/ServicioSocial/blob/main/ENTREGABLES/Deep%20Learning/Mapa%20Puebla/WideTrainingExample_Puebla.ipynb

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed.). Springer.

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366.

Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

Csáji, B. C. (2001). *Approximation with Artificial Neural Networks*. Tesis de Maestría, Facultad de Ciencias, Universidad Eötvös Loránd, Hungría.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232.